Smell-Ranker: Infrastructure & Architecture Documentation

Project: Automated Code Smell Prioritization and Ranking **Version:** 0.1 (Phase 1 - Component Validation)

1. System Overview

Smell-Ranker is an automated pipeline designed to generate a "Fortified Ground Truth" for code smell prioritization. It achieves this by mining software repositories to extract objective developer actions (refactoring, bug-fixing) and using them to score code smells detected by static analysis.

The system operates on a **Hybrid Workflow** that leverages three distinct environments to ensure persistence, scalability, and reproducibility.

The Hybrid Workflow

The architecture splits responsibilities across three layers:

1. **Persistent Storage (Google Drive):** Holds the "state" of the project (tools, input data, results).
2. **Development Environment (Local/GitHub):** Where the logic is written and versioned.
3. **Runtime Environment (Google Colab):** A disposable compute engine that executes the logic.

2. File Structure & Organization

A. Local Development (`smell-ranker/`)

This is the source of truth for all code. It is version-controlled on GitHub.

```
smell-ranker/
├── pipeline/                # The main Python Application Package
│   ├── bin/                 # Executable Shell Scripts (Entry Points)
│   │   ├── exec_pipeline.sh  # MASTER SCRIPT: The single command to run the whole expe
│   │   ├── colab_git_setup...# SYNC SCRIPT: Handles secure Git cloning/pulling
│   │   └── colab_env_setup.sh# SETUP SCRIPT: Installs Java 17 & Mounts Drive
│   │
│   ├── utils/               # Python Utility Package
│   │   ├── cmd_runner.py     # Adapter for running shell commands safely
│   │   └── __init__.py       # Exposes utilities to the app
│   │
│   ├── main.py              # FACADE: The main Python entry point. Orchestrates the p
│   ├── config.py            # CONFIG: Holds all paths (Drive, Tools) and settings.
│   ├── repo_metrics.py       # ANALYSIS: Calculates project stats (LOC, Commits) for v
│   └── __init__.py          # Package marker
│
├── notebooks/               # Jupyter Notebooks for Colab Control
│   ├── 01_project_setup.ipynb      # One-Time Setup & Verification Dashboard
│   └── 02_pipeline_execution.ipynb # Daily Execution Trigger
│
└── docs/                    # Project Documentation
```

```
    └── infrastructure_architecture.md
```

B. Persistent Storage (Google Drive: `Thesis_Project/`)

This folder structure is manually created once and persists across Colab sessions.

```
Thesis_Project/
├── tools/                    # External Binaries
│   ├── pmd-bin-7.18.0/       # PMD Static Analyzer (Pre-downloaded)
│   └── RefactoringMiner_v3/  # RefactoringMiner (Built from source via Gradle)
│
├── repos/                    # Input Data
│   └── toy_project/          # 'refactoring-toy-example' (Cloned & Verified)
│
├── scripts/                  # Code Sync Target
│   └── (The 'smell-ranker' repo is cloned here by the execution script)
│
└── outputs/                  # Experiment Results
    ├── refactorings.json     # Output from Pass 1
    └── pmd_report.xml        # Output from Pass 2
```

3. Execution Flow (The "How-To")

The system is designed to be run from **Google Colab**, triggered by the `02_pipeline_execution.ipynb` notebook.

Step 1: Initialization

- **Trigger:** User runs `!bash exec_pipeline.sh` in Colab.
- **Action:** `exec_pipeline.sh` calls `colab_env_setup.sh` .
- **Result:** Google Drive is mounted at `/content/drive` , and `openjdk-17-jdk` is installed via `apt-get` .

Step 2: Synchronization

- **Trigger:** `exec_pipeline.sh` calls `colab_git_setup_smell_ranker.sh` with the Git URL.
- **Action:** The script checks if `Thesis_Project/scripts/` contains a git repo.
  - *If No:* It runs `git clone` using the injected secure token.
  - *If Yes:* It runs `git pull` to get the latest code from GitHub.
- **Result:** The local Python code in Drive is identical to your PyCharm code.

Step 3: Execution

- **Trigger:** `exec_pipeline.sh` calls `python3 -m pipeline.main` .

- **Action:** Python starts. `main.py` imports `config.py` to get the paths.
- **Process:**
  1. **Phase 1 (Smoke Test):** Checks connectivity to RefactoringMiner and PMD by printing their execution strings.
  2. *(Future Phase 2):* Will execute the "Fast Scan" using RM.
  3. *(Future Phase 3):* Will execute the "Slow Analysis" using PMD on checked-out commits.

4. Design Principles & Patterns

The architecture strictly adheres to software engineering best practices to ensure the tool is defensible in a thesis context.

Principle	Implementation
Separation of Concerns (SoC)	Logic is separated from Configuration (`config.py`). Execution (`bin/`) is separated from Application (`pipeline/`).
Single Responsibility (SRP)	Each shell script does exactly one thing (Env, Sync, or Execute).
Facade Pattern	`main.py` and `exec_pipeline.sh` act as simple "front doors" that hide the complex subsystem logic.
Adapter Pattern	`cmd_runner.py` adapts the raw shell interfaces of PMD/RefactoringMiner into clean Python functions.
DRY (Don't Repeat Yourself)	Environment setup logic is centralized in one script (`colab_env_setup.sh`) used by all notebooks.

5. Toolchain Configuration

RefactoringMiner

- **Version:** 3.0 (Built from Source)
- **Build Requirement:** Java 17 (JDK)
- **Role:** Pass 1 - History Mining. Identifies refactoring operations in the commit log.

PMD

- **Version:** 7.18.0 (Binary Distribution)
- **Role:** Pass 2 - Candidate Generation. Analyzes static code snapshots to find *God Class*, *Long Method*, and *Feature Envy*.
- **Justification:** Chosen over JDeodorant for feasibility and command-line compatibility.

6. Future Roadmap

- **Phase 2 Implementation:** Implement `pass_1_fast_scan.py` to parse RefactoringMiner JSON output.
- **Phase 3 Implementation:** Implement `pass_2_slow_analysis.py` to handle `git checkout` and run PMD loops.
- **Phase 4 Implementation:** Implement `heuristics.py` to score the overlap between Pass 1 and Pass 2 data.